Datos Cósmicos

Descubre Modelos y Validación de Pydantic

Resumen: Domina la validación de datos de Pydantic a través de ejercicios con temática espacial. Aprende a crear modelos robustos, implementar validación personalizada y manejar estructuras anidadas mientras gestionas flujos de datos cósmicos.

Versión: 3.0

Contenido

I Introducción **2**

II Instrucciones de IA **3**

III Instrucciones Generales **5**

III.1 Requisitos de Python 5
III.2 Estructura del Proyecto 5
III.3 Herramientas Disponibles 6
III.4 Conceptos Clave de Pydantic 7

IV Ejercicio 0: Datos de la Estación Espacial **8**

IV.1 Contexto 8
IV.2 Requisitos 8

V Ejercicio 1: Registros de Contacto Alienígena **10**

V.1 Contexto 10
V.2 Requisitos 10

VI Ejercicio 2: Gestión de la Tripulación Espacial **13**

VI.1 Contexto 13
VI.2 Requisitos 13

VII Entrega y revisión por pares **16**

Capítulo I

Introducción

Bienvenido al Observatorio de Datos Cósmicos, la principal instalación de procesamiento de datos de la galaxia. ¡Como Ingeniero de Datos junior, se te ha asignado validar flujos de datos entrantes de diversas misiones espaciales, informes de contacto con alienígenas y sistemas de monitoreo de estaciones.

Tu misión: aprender Pydantic; la biblioteca de validación de datos más poderosa de Python; a través de escenarios cósmicos prácticos. Progresarás desde modelos básicos hasta reglas de validación complejas, asegurando la integridad de los datos a lo largo del universo conocido.

El Tema Cósmico

A lo largo de esta actividad, trabajarás con datos con temática espacial que facilitan el aprendizaje, involucrando:

- Estaciones Espaciales: fundamentos básicos de validación de datos
- Contactos Alienígenas: reglas y lógica de validación personalizadas
- Gestión de tripulación: modelos anidados y relaciones complejas

Cada ejercicio se basa en el anterior, introduciendo conceptos de Pydantic de forma progresiva.

Consejo profesional: Pydantic convierte automáticamente tipos compatibles (como cadenas a datetime) y proporciona mensajes de error detallados cuando la validación falla. Esto lo hace perfecto para construir APIs y sistemas de procesamiento de datos robustos!

Capítulo II

Instrucciones de IA

Contexto

Durante tu viaje de aprendizaje, la IA puede ayudar con muchas tareas diferentes. Tómate el tiempo para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, abórdalas siempre con precaución y evalúa críticamente los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca puedes estar completamente seguro de que tu pregunta estuvo bien formulada o de que el contenido generado sea preciso. Tus compañeros son un recurso valioso para ayudarte a evitar errores y puntos ciegos.

Mensaje principal

- Utiliza IA para reducir tareas repetitivas o tediosas.
- Desarrolla habilidades de prompting tanto de codificación como de no codificación que beneficiarán tu futura carrera.
- Aprende cómo funcionan los sistemas de IA para anticipar y evitar mejor los riesgos, sesgos y problemas éticos comunes.
- Sigue desarrollando habilidades técnicas y de alto rendimiento trabajando con tus pares.
- Solo usa contenido generado por IA que entiendas completamente y por el que puedas asumir la responsabilidad.

Reglas del aprendiz:

- Debes tomarte el tiempo para explorar las herramientas de IA y entender cómo funcionan, para que puedas usarlas de manera ética y reducir posibles sesgos.
- Debes reflexionar sobre tu problema antes de dar indicaciones: esto te ayuda a escribir indicaciones más claras, detalladas y relevantes usando un vocabulario preciso.
- Debes desarrollar el hábito de revisar, cuestionar y probar de forma sistemática cualquier cosa generada por IA.
- Siempre busca revisión de pares no te la juegues confiando únicamente en tu validación.

Resultados de la fase:

- Desarrolla habilidades de prompting tanto de uso general como específico de dominio.
- Aumenta tu productividad con un uso eficaz de herramientas de IA.
- Continúa fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

Comentarios y ejemplos:

- Con frecuencia te encontrarás con situaciones exámenes, evaluaciones y más donde debes demostrar una comprensión real. Prepárate, sigue desarrollando tanto tus habilidades técnicas como interpersonales.
- Explicar tu razonamiento y debatir con compañeros a menudo revela lagunas en tu comprensión. Prioriza el aprendizaje entre pares.
- Las herramientas de IA a menudo carecen de tu contexto específico y tienden a proporcionar respuestas genéricas. Tus compañeros, que comparten tu entorno, pueden ofrecer ideas más relevantes y precisas.
- Donde la IA tiende a generar la respuesta más probable, tus compañeros pueden proporcionar perspectivas alternativas y matices valiosos. Confía en ellos como un punto de control de calidad.

✓ Buenas prácticas:

> Pido a la IA: “¿Cómo pruebo una función de ordenamiento?” Me da algunas ideas. Las pruebo y reviso los resultados con un compañero. Refinamos el enfoque juntos.

✗ Malas prácticas:

> Pido a la IA que escriba toda una función, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad, y fracaso mi proyecto.

✓ Buenas prácticas:

> Uso la IA para ayudar a diseñar un analizador. Luego reviso la lógica con un compañero. Detectamos dos errores y lo reescribimos juntos: mejor, más limpio y completamente entendido.

✗ Malas prácticas:

> Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no logro justificarlo y fracaso en mi proyecto.

Capítulo III
Instrucciones Generales

III.1 Requisitos de Python

- Tu proyecto debe estar escrito en Python 3.10 o posterior.
- Tu proyecto debe cumplir con el estándar de codificación flake8.
- Todo el código debe incluir anotaciones de tipo completas. Verifícalo usando mypy.
- El manejo de excepciones debe proteger los flujos de datos de la corrupción.
- Todas las clases y colecciones estándar están autorizadas, junto con sus métodos (int, str, list, dict, etc.).
- Todas las funciones integradas están autorizadas.
- Utiliza pip como gestor de paquetes
- Debes usar entornos virtuales (recomendado: venv, virtualenv o conda)
- Debes usar Pydantic 2.x en cada ejercicio. (Debe instalarse mediante pip). Solo otros módulos serán listados en la sección Permitidos de cada ejercicio

III.2 Estructura del Proyecto

- Cada ejercicio debe completarse en su propio directorio: ex0/, ex1/, ex2/
- Tu código debe demostrar comprensión de los conceptos, no solo soluciones copiadas y pegadas
- Todos los ejercicios se construyen uno sobre otro; complétalos en orden
- Documenta tu proceso de aprendizaje mediante comentarios claros en el código y nombres de variables significativos
- Envía tu trabajo a tu repositorio Git asignado

III.3 Herramientas disponibles

Esta actividad incluye herramientas de generación de datos para ayudarte a probar tus modelos de Pydantic:

- data_generator.py - Generar datos de prueba realistas para todos los ejercicios
- data_exporter.py - Exportar datos en formatos JSON, CSV y Python
- generated_data/ - Conjuntos de datos pregenerados listos para usar

Importaciones autorizadas: Puede importar datos JSON y CSV desde las herramientas directorio. Los módulos de la biblioteca estándar (json, csv, datetime, etc.) están permitidos.

Importante: Esta actividad se centra en la sintaxis de Pydantic v2. Evita decoradores obsoletos como @validator: utiliza @model_validator para validación personalizada en su lugar.

III.4 Conceptos clave de Pydantic

BaseModel

La base de todos los modelos Pydantic. Hereda de BaseModel para crear clases de datos validadas.

Campo

Utiliza Field(...) para añadir restricciones de validación, descripciones y valores predeterminados a los atributos del modelo.

model_validator

Utiliza el decorador @model_validator(mode='after') para una lógica de validación personalizada que se ejecuta después de la validación integrada de Pydantic. Esto reemplaza al obsoleto decorador @validator de Pydantic v1.

Capítulo IV

Ejercicio 0: Datos de la Estación Espacial

	Ejercicio0
	Datos de la Estación Espacial
Directorio: ex0/	
Archivos a Enviar: space_station.py	
Autorizado: Ninguno	

Objetivo: Validación Aprender la creación básica de modelos Pydantic con BaseModel y **de campos.**

IV.1 Antecedentes

La Observatorio de Datos Cósmicos monitorea cientos de estaciones espaciales a través de la galaxia. Cada estación informa estadísticas vitales que incluyen tamaño de la tripulación, niveles de energía y estado operativo. Su primera tarea es crear un sistema de validación para estos datos críticos.

IV.2 Requisitos

Modelo SpaceStation

Cree un modelo Pydantic con estos campos validados:

- station_id: Cadena, 3-10 caracteres
- nombre: Cadena, 1-50 caracteres
- tamaño_de_tripulación: Entero, 1-20 personas

- nivel_poder: Flotante, 0.0-100.0 por ciento
- nivel_de_oxígeno: Flotante, 0.0-100.0 por ciento
- mantenimiento_l ultar: Campo DateTime
- está_operativo: Booleano, predeterminado a True
- notas: Cadena opcional, máximo 200 caracteres

Función de demostración

Incluye una función main() que:

- Crea una instancia válida de estación espacial
- Muestra la información de la estación claramente
- Intenta crear una estación inválida (p. ej., tamaño de tripulación > 20)
- Muestra el mensaje de error de validación

Ejemplo de salida esperada:

```
Validación de datos de la estación espacial
==========================================
Estación válida creada:
ID: ISS001
Nombre: Estación Espacial Internacional
Tripulación: 6 personas
Potencia: 85.5%
Oxígeno: 92.3%
Estado: Operativa

==========================================
Error de validación esperado: la entrada debe ser
menor o igual a 20
```

Piensa en: ¿Cómo funciona la conversión automática de tipos de Pydantic? ¿Qué ocurre cuando pasas una marca de tiempo en forma de cadena a un campo datetime?

Capítulo V

Ejercicio 1: Registros de Contacto Alienígena

	Ejercicio1
Registros de Contacto Alienígena	
Directorio: ex1/	
Archivos a enviar: alien_contact.py	
Autorizado: None	

Objetivo: Dominio de validación personalizada usando @model_validator para reglas de negocio complejas.

V.1 Antecedentes

El Observatorio recibe informes de contacto alienígena de toda la galaxia. Estos informes sensibles requieren reglas de validación sofisticadas que van más allá de simples restricciones de campo. Diferentes tipos de contacto tienen diferentes requisitos, y ciertas combinaciones de datos indican informes potencialmente fraudulentos.

V.2 Requisitos

Enum de Tipo de Contacto

Define tipos de contacto: radio, visual, físico, telepático

Modelo de AlienContact

Crea un modelo de Pydantic con estos campos:

- contact_id: Cadena, 5-15 caracteres
- marca de tiempo: DateTime del contacto
- ubicación: Cadena, 3-100 caracteres
- tipo_de_contacto: enum ContactType
- fuerza_de_senal: Flotante, escala 0.0-10.0
- duracion_en_minutos: Entero, 1-1440 (máx. 24 horas)
- cuenta_de_testigos: Entero, 1-100 personas
- mensaje_recibido: Cadena opcional, máx. 500 caracteres
- est_verificado: Booleano, por defecto False

Reglas de validación personalizadas

Implementa @model_validator(mode='after') con estas reglas de negocio:

- El ID de contacto debe comenzar con "AC" (Alien Contact)
- Los informes de contacto físico deben estar verificados
- El contacto telepático requiere al menos 3 testigos
- Señales fuertes (> 7.0) deben incluir mensajes recibidos

Función de demostración

Muestra informes de contacto válidos e inválidos con mensajes de error claros.

Ejemplo de salida esperada:

```
Validación de registro de contacto
alienígena
========================================
Informe de contacto válido: ID:
AC_2024_001 Tipo: radio Ubicación: Area
51, Nevada Señal: 8.5/10 Duración: 45
minutos Testigos: 5 Mensaje: 'Saludos
desde Zeta Reticuli'


========================================
Error de validación esperado: El contacto telepático requiere
al menos 3 testigos
```

Consejo Avanzado: El decorador @model_validator te permite validar todo el modelo después de que se hayan validado todos los campos. Recuerda devolver self al final de tu función validadora.

Capítulo VI

Ejercicio 2: Gestión de la tripulación espacial

	Ejercicio2
Gestión de la tripulación espacial	
Directorio: ex2/	
Archivos para enviar: space_crew.py	
Autorizado: Ninguno	

Objetivo: Domina modelos Pydantic anidados y datos complejos **relaciones.**

VI.1 Antecedentes

Las misiones espaciales requieren una gestión minuciosa de la tripulación. Cada misión tiene varios miembros de la tripulación con diferentes rangos, especialidades y niveles de experiencia. El Observatorio debe validar que las tripulaciones de la misión cumplan con los requisitos de seguridad y operativos antes de la aprobación del lanzamiento.

VI.2 Requisitos

Enum de Rangos

Define rangos de la tripulación: cadete, oficial, teniente, capitán, comandante

Modelo CrewMember

Miembro individual de la tripulación con estos campos:

- id_miembro: Cadena, 3-10 caracteres
- nombre: Cadena, 2-50 caracteres
- rango: enumeración Rank
- edad: Entero, 18-80 años
- especialización: Cadena, 3-30 caracteres
- años_de_experiencia: Entero, 0-50 años
- está_activo: Booleano, por defecto True

Modelo SpaceMission

Misión con lista de tripulación y estos campos:

- id_misión: Cadena, 5-15 caracteres
- nombre_misión: Cadena, 3-100 caracteres
- destino: Cadena, 3-50 caracteres
- fecha_lanzamiento: FechaHora
- días_duración: Entero, 1-3650 días (máximo 10 años)
- tripulación: Lista de CrewMember, 1-12 miembros
- estado_misión: Cadena, por defecto a "planeado"
- presupuesto_millones: Flotante, 1.0-10000.0 millones de dólares

Reglas de Validación de Misión

Implemente @model_validator(mode='después') con estos requisitos de seguridad:

- El ID de Misión debe empezar con "M"
- Debe haber al menos un Comandante o Capitán
- Misiones largas (> 365 días) requieren 50% tripulación experimentada (5+ años)
- Todos los miembros de la tripulación deben estar activos

Función de demostración

Muestra una misión válida con detalles de la tripulación y una misión inválida que falla la validación.

Ejemplo de salida esperada:

```
Validación de la tripulación de la misión espacial
==========================================
Misión válida creada:
Misión: Establecimiento de la colonia en Marte
ID: M2024_MARS
Destino: Marte
Duración: 900 días
Presupuesto: $2500.0M
Tamaño de la tripulación: 3
Miembros de la tripulación:
- Sarah Connor (comandante) - Mando de la misión
- John Smith (teniente) - Navegación
- Alice Johnson (oficial) - Ingeniería

==========================================
Error de validación esperado: la misión debe tener al menos un
Comandante o Capitán
```

Piensa en ello: ¿Cómo maneja Pydantic la validación de anidados ¿modelos? ¿Qué sucede cuando un Miembro de la tripulación falla en la validación dentro de una Misión Espacial?

Capítulo VII

Presentación y revisión por pares

Envía tu tarea en tu repositorio de Git como de costumbre. Solo el trabajo dentro de tu repositorio será evaluado durante la defensa. No dudes en verificar dos veces los nombres de tus archivos para asegurarte de que sean correctos.

Solo necesitas devolver los archivos solicitados por la asignatura de esta actividad.